iRAYPLE Industrial Camera

SDK Development Document (C# Interface)



Revision History

No.	Date	Version	Revision Content	Software Version
1	2023/03	V1.0	Update Content	2.4.0
2	2024/04	V1.1	Added Save Image Function	2.4.2
3	2025/11	V1.2	Added MVPImage Construction Method	2.6.1

Table of Contents

1 Overview 5
1.1 Compatibility Introduction 5
1.2 SDK Installation Directory 5
1.2.1 SDK for Windows 5
1.2.2 SDK for Linux 7
1.3 Related Resources of SDK (C#) 7
1.4 SDK Engineering Configuration 8
1.4.1 VS2013 ~ VS2022 8
2 Overall Call Process of SDK 11
3 Function Interface Calling of the SDK 12
3.1 System Operation Instruction 12
3.1.1 Obtain Version Information 12
3.1.2 Device Enumeration (Find Device) 12
3.1.3 Handle Creation and Destruction of Device 13
3.2 Camera Operation Instruction 14
3.2.1 Device Open and Close 14
3.2.2 Camera Configuration Download, Storage and Loading 15
3.2.3 Camera Configuration Settings (Only for GigE Camera) 16
3.2.4 Unique Features 17
3.3 Register Event Callback 18
3.4 Camera Stream Resource Operation Instruction 20
3.4.1 Stream Data Configuration 20
3.4.2 Image Collection 22
3.4.3 Acquire ImageData 23
3.4.4 Operations on Streaming and Image Data 24
3.5 Attribute Operation Instruction 25
3.5.1 Determination, Acquisition and Setting of Properties 25
3.5.2 Execution of Command Attribute 31
3.5.3 Attribute Search and Usage 32
3.6 Image Operation 40
3.6.1 Video Recording 40
3.6.2 Image Operation 41
4 Conversion method for Image Format of 3rd-party Platform 43
4.1 Halcon 43
4.1.1 Mono8 (Monochrome Image) 43
4.1.2 Non-Mono8 (Color Image) 43
4.2 MvpImage Object 44
4.2.1 Mono8 (Monochrome Image) 44
4.2.2 Non-Mono8 (Color Image) 44
5 Matched Routine Description 45
6 FAQ (Frequently Asked Question) 46
6.1 Only 8 Images are Obtained by Using the C# Interface 46

1 Overview

1.1 Compatibility Introduction

Table 1-1 Operating System Requirement

OS Type	Version	Note
Windows *1	Win7 SP1 32/64bit	Ensure that OS version includes the patch of SP1.
	Win8 32/64bit	
	Win10 32/64bit	
Linux (x86) *2	**System Version**: Ubuntu 12.04 LTS ~ Ubuntu 21.04 LTS (32-bit/64-bit)	The upgraded kernel version system is excluded.
	CentOS 6.5 ~ CentOS 8.0 (32-bit/64-bit)	The upgraded kernel version system is excluded.

- Windows XP is no longer supported since SDK V2.2.5.
- The supported kernel version of Linux is 2.6.32 to 5.11.0 (Inclusive).
- Use the command of **uname –a** to check the kernel version number.

1.2 SDK Installation Directory

1.2.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

In V2.1.0, user can select the installation path.

Figure 1-1 Installation directory structure

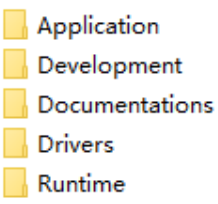


Table 1-2 Installation directory structure description

Application	● The client software is compatible with 32-bit and 64-bit. ● The matched tool is compatible with 32-bit and 64-bit.
Development	**3rdPartyPlatformAdapter** It includes two plug-in dynamic libraries of third-party algorithm platform (Halcon and Sherlock).
	DOTENT DLL It includes **.Net** library file which is imported when user create the project.
	Include It includes the header file of the SDK interface, which explains the basic properties of various interfaces. The specific usage of header files is in the **samples folder**. The other is the extension definition of the header file of the SDK, which defines the SDK error code, the user-defined parameters, the data restriction of some parameters, the enumeration of various behaviors, and the data structure of various information.
	Lib It includes the lib files.
	Samples It includes the various types of demo file, such as **C#**, **Delphi**, **Lab View**, **MFC**, **Python**, **QT**, **VB** and **VC**.
Documentations	● **.Net** interface document ● **ActiveX** interface document ● **C** interface document ● **SDK** development document
Drivers	● The installation and uninstallation software of **DShowFilter**. ● The installation and uninstallation software of **GigE** high-performance driver. ● The installation and uninstallation software of **PCIex86** smart camera driver. ● The installation and uninstallation software of **USB** high-performance driver.
Runtime	The compilation dynamic link library when running the SDK.

	It includes the third-party **GenICam** library which is separated into two folders as 32-bit and 64-bit.

1.2.2 SDK for Linux

The installation directory and introduction of SDK for Linux is as follows:

Figure 1-2 folder structure description

```
MVviewer
 ├ bin        : GUI Tool, discover/connect camera, image acquisition, properties setting, uninstall script and etc.;
 ├ include    : SDK head files, image convert head files;
 ├ lib        : SDK dll, third-party library, image converting lib;
 ├ module
 |  ├ GigEDriver : GigE camera driver;
 |  └ USBDriver  : USB camera driver;
 └ share
    ├ C       : C samples;
    ├ Qt      : Qt samples;
    └ Python  : Python samples;
```

1.3 Related Resources of SDK (C#)

Table 1-3 SDK source description

Category	Item	File Storage Folder	Name
SDK Library	DOTNET_3.5 (32-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMV\DOTNET_3.5\x86	MVSDK_Net.dll
	DOTNET_3.5 (64-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMV\DOTNET_3.5\x64	MVSDK_Net.dll
	DOTNET_4.0 (32-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMV\DOTNET_4.0\x86	MVSDK_Net.dll
	DOTNET_4.0 (64-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMV\DOTNET_4.0\x64	MVSDK_Net.dll
Development Document	Chinese	【SDK Installation Directory】\Documentations\	IMV_API_Net_CHS.chm iRAYPLE Industrial Camera SDK Development Document (C# Interface)

	English	【SDK Installation Directory】\ Documentations\	IMV_API_Net_ ENG.chm
Routine	—	【SDK Installation Directory】 \Development\Sample\C#\IMV	

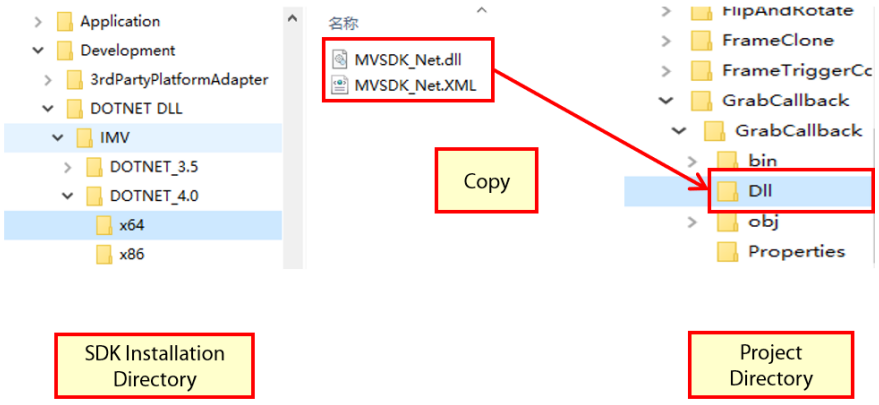
1.4 SDK Engineering Configuration

1.4.1 VS2013 ~ VS2022

Take DOTNET4.0 64-bit as an example to demonstrate the method of project configuration.

Step 1 Copy the related sources of SDK into the project directory.

Figure 1-3 Project configuration



Project Configuration

Step 2 Click Project > Add Reference > Add project file > OK.

Figure 1-4 Add reference

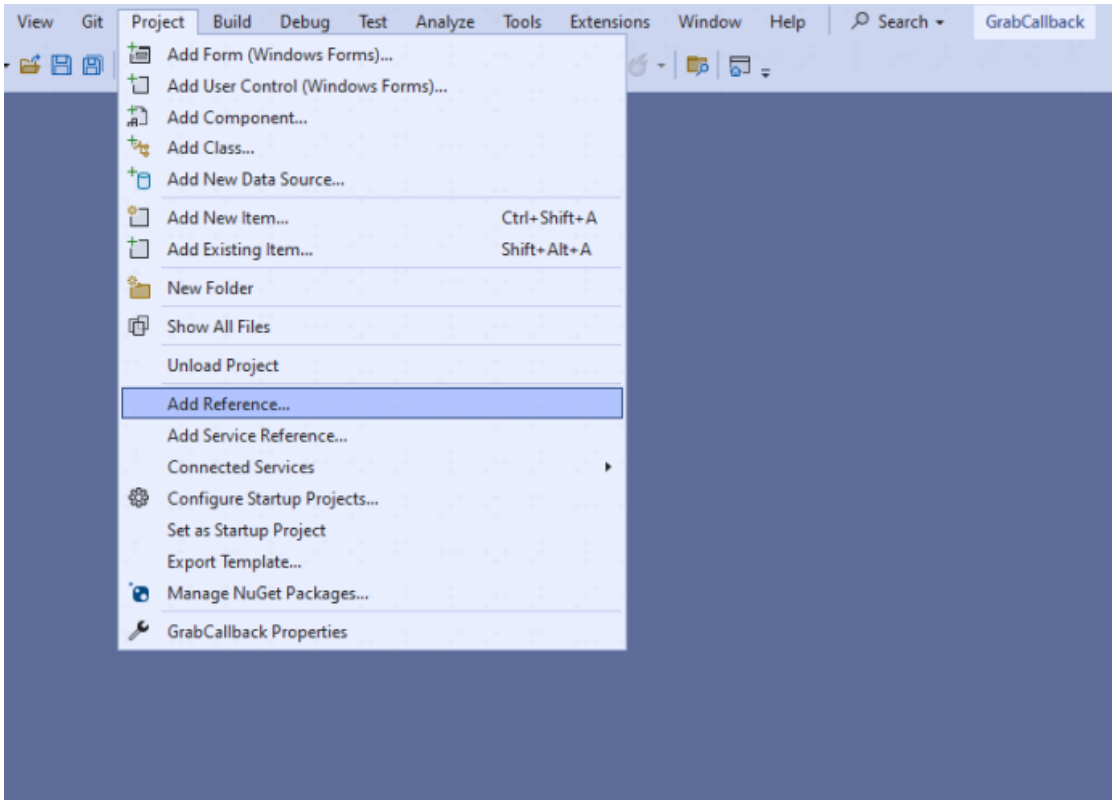
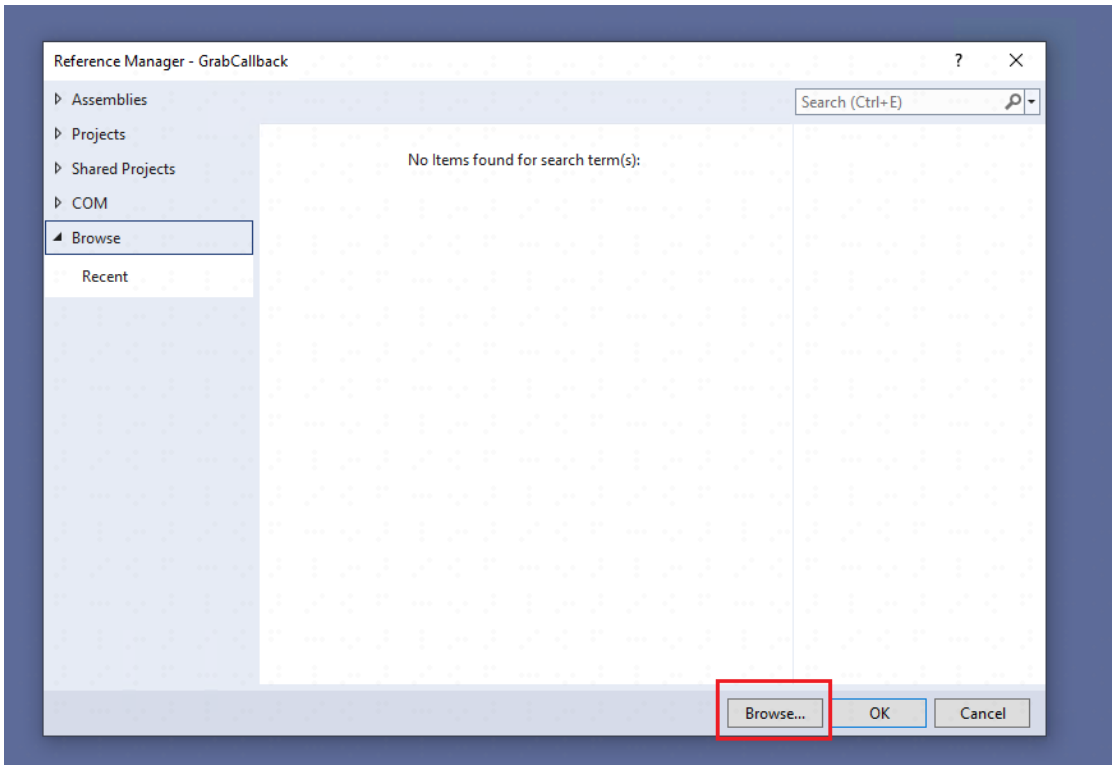
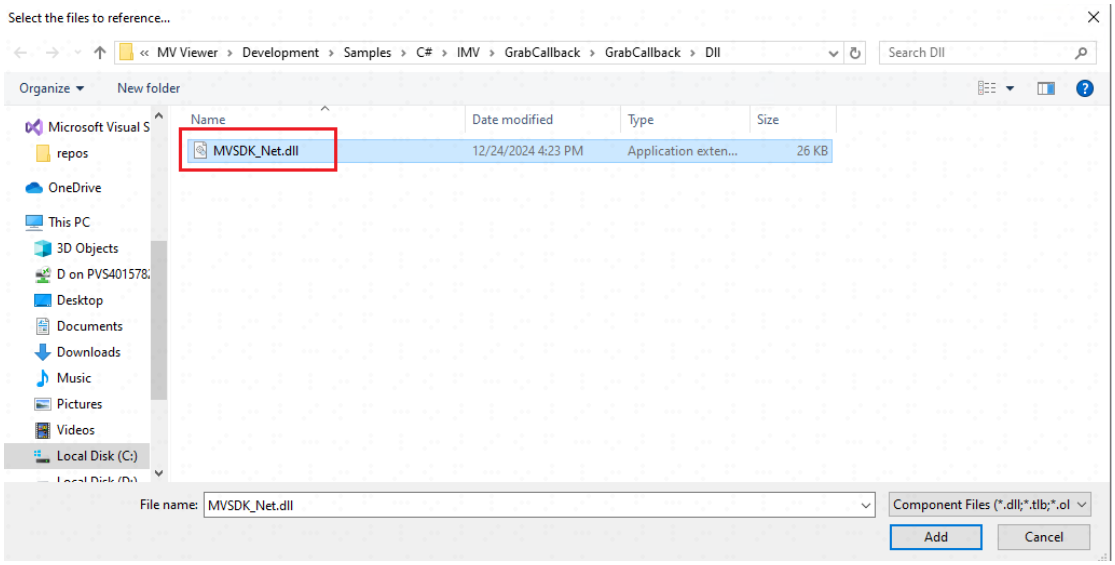
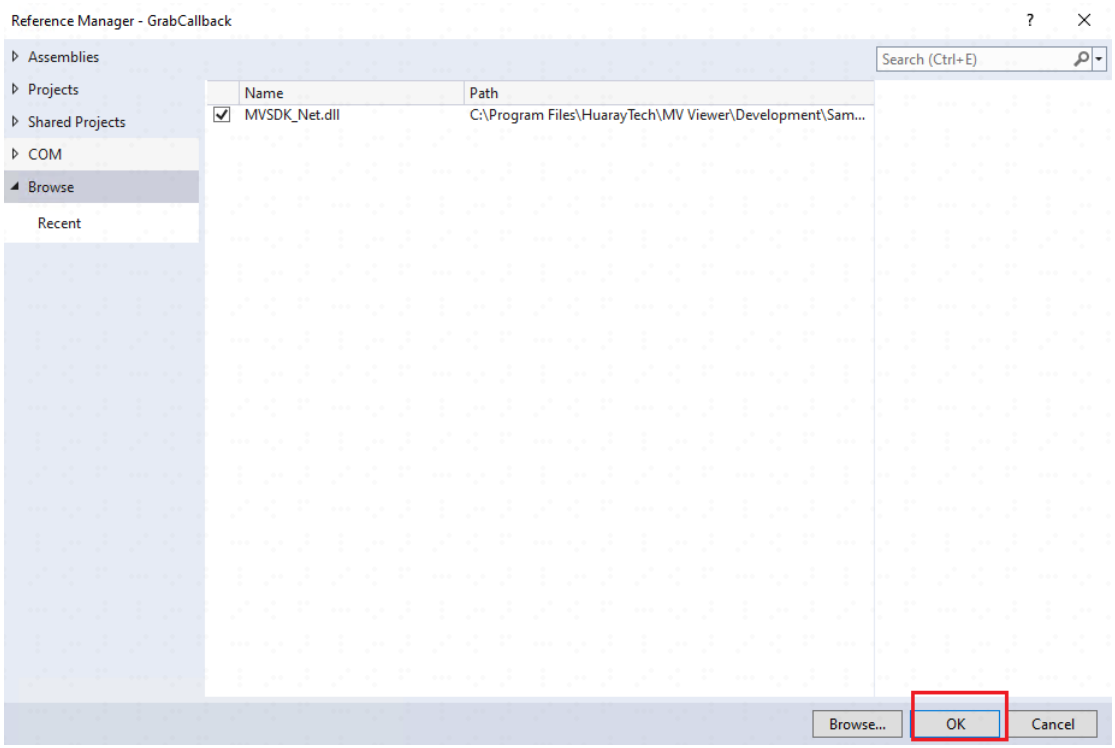


Figure 1-5 Add project file







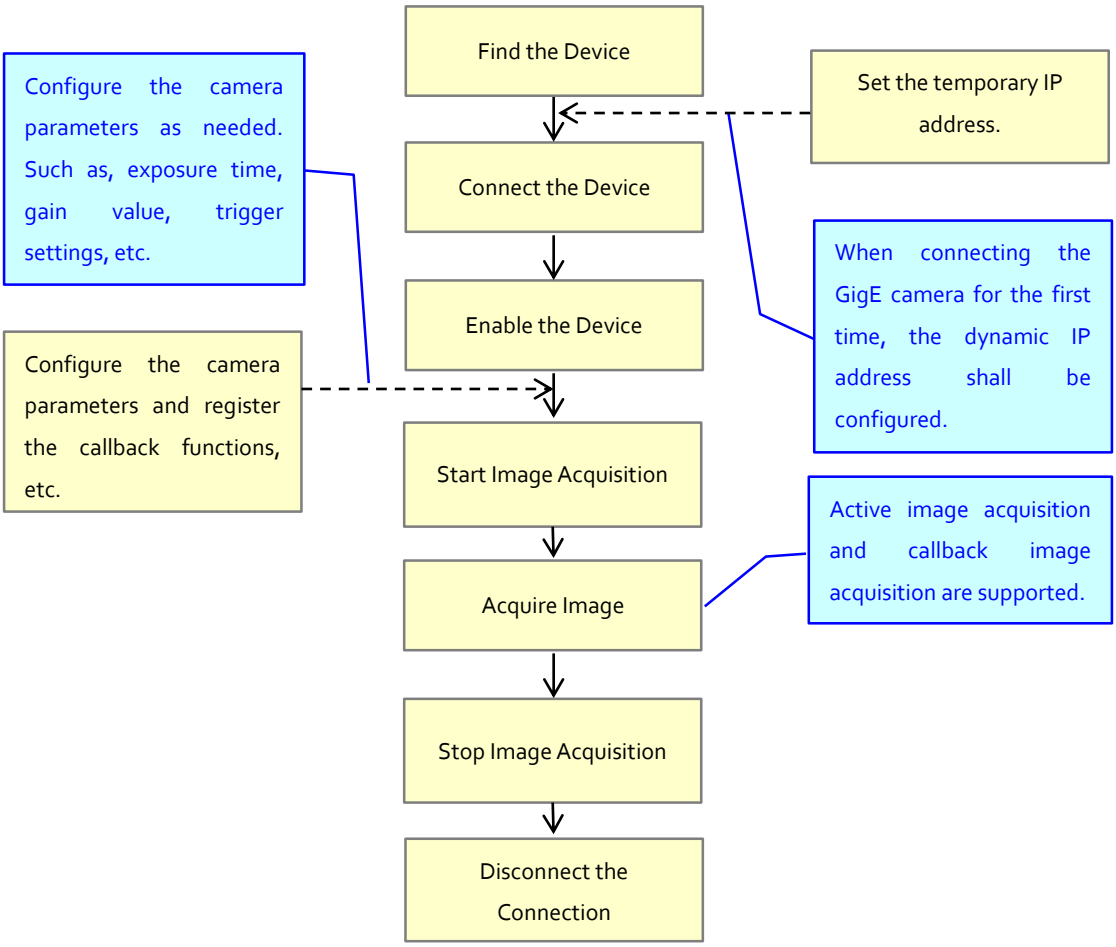
Step 3 Write-in the using **MVSDK_Net** in the calling code.

Figure 1-6 Add calling code

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Runtime.InteropServices;
5  using System.Text;
6  using System.Threading;
7  using System.Threading.Tasks;
8  using MVSDK_Net;
```

2 Overall Call Process of SDK

Figure 2-1 SDK calling process



3 Function Interface Calling of the SDK

3.1 System Operation Instruction

3.1.1 Obtain Version Information

- Interface: public static string IMV_GetVersion()
  - ◇ Description: Get OS version. If succeeded, return version information. If failed, return **NULL**.
  - ◇ Reference Routine: All current routines can be referenced.

3.1.2 Device Enumeration (Find Device)

- Interface: public static int IMV_EnumDevices(ref IMVDefine.IMV_DeviceList pDeviceList, UInt32 interfaceType)
  - ◇ Description: Device enumeration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-1 Parameter description

Parameter	Description
pDeviceList [OUT]	Device list.
interfaceType[IN]	The interface type to be enumerated can be combined arbitrarily, such as **interfaceTypeGige | interfaceTypeUsb3**.

- □ When **interfaceType = interfaceTypeAll**, enumerate all online devices connected through different interfaces.
- □ When **interfaceType = interfaceTypeGige**, enumerate all online devices connected through the Gige interface.
- □ When **interfaceType = interfaceTypeUsb3**, enumerate all online devices connected through the USB interface.
- □ When **interfaceType = interfaceTypeCL**, enumerate the online devices connected through the **CameraLink** interface.
- □ The **interfaceType** of the **IMV_EnumDevices** support any combination of interface types. For example, if you need to enumerate all online devices connected through the GigE port and USB 3.0 port, the **interfaceType** can

be written as the **interfaceType = interfaceTypeGige | interfaceTypeUsb3**.

◇ Reference Routine: All current routines can be referenced.

- Interface: public static int IMV_EnumDevicesByUnicast(ref IMVDefine.IMV_DeviceList pDeviceList, string pIpAddress);

  ◇ Description: enumerate devices in unicast format (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-2 Parameter description

Parameter	Description
pDeviceList [OUT]	Device list.
pIpAddress [IN]	The IP address of the device.

  ◇ Reference Routine: **UnicastMode**.

3.1.3 Handle Creation and Destruction of Device

- Interface: public int IMV_CreateHandle(IMVDefine.IMV_ECreateHandleMode mode, int cameraIndex = 0, string cameraStr = "");

  ◇ Description: Create the handle by specifying the identifier, such as the specified index, device key, custom device name and IP address. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-3 Parameter description

Parameter	Description
mode[IN]	The creation mode of device.
cameraIndex [IN]	Index ID
cameraStr[IN]	Device key, device custom name and IP address.

  ◇ Reference Routine: All current routines can be referenced.

- Interface: public int IMV_DestroyHandle();

  ◇ Description: Destroy the handle of device. If succeeded, return **IMV_OK**. If failed, return error code.

  ◇ Reference Routine: All current routines can be referenced.

3.2 Camera Operation Instruction

- Interface: public int IMV_GetDeviceInfo(ref IMVDefine.IMV_DeviceInfo pDevInfo);
  - ◇ Description: Obtain device information. If succeeded, return IMV_OK. If failed, return error code.

Table 3-4 Parameter description

Parameter	Description
pDevInfo [OUT]	Device information.

  - ◇ Reference Routine: All current routines can be referenced.

3.2.1 Device Open and Close

- Interface: public int IMV_Open();
  - ◇ Description: Open device. If succeeded, return **IMV_OK**. If failed, return error code.
  - ◇ Reference Routine: All current routines can be referenced.
- Interface: public int IMV_OpenEx(IMVDefine.IMV_ECameraAccessPermission accessPermission);
  - ◇ Description: Open device. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-5 Parameter description

Parameter	Description
accessPermission[IN]	Control the channel permission (the **accessPermissionControl** is adopted by default in **IMV_Open**).

    - □ When using the **IMV_OpenEx** to execute the connection operation, the access rights will be determined.
  - ◇ Reference Routine: None.
- Interface: public bool IMV_IsOpen();
  - ◇ Description: Determine whether the device is opened. If opened, return **True**. If close or offline, return **False**.
  - ◇ Reference Routine: **ForceIp**.
- Interface: public int IMV_Close();
  - ◇ Description: Close device. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

3.2.2 Camera Configuration Download, Storage and Loading

- Interface: public int IMV_DownLoadGenICamXML(string pFullFileName);
  - ◇ Description: Download the device description XML file, and save it to the specified path, such as **D:\\xml.zip**. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-6 Parameter description

Parameter	Description
pFullFileName[IN]	File saving path.

  - ◇ Reference Routine: None.
- Interface: public int IMV_SaveDeviceCfg(string pFullFileName);
  - ◇ Description: Saving the device configuration file to the specified folder. The file with the same name that already exists will be overwritten. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-7 Parameter description

Parameter	Description
pFullFileName [IN]	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.

  - ◇ Reference Routine: **LoadAndSaveDeviceCfg**.
- Interface: public int IMV_LoadDeviceCfg(string pFullFileName, ref IMVDefine.IMV_ErrorList pErrorList);
  - ◇ Description: Load the xml configuration of device from files. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-8 Parameter description

Parameter	Description
pFullFileName [IN]	Full name (including saving path) of the device configuration file (XML), such as **D:\\config.xml** or **D:\\config.mvcfg**.
pErrorList [OUT]	The list of attribute name that failed to load. The upper limit of attributes that fails to save and load is **IMV_MAX_ERROR_LIST_NUM**.

◇ Reference Routine: **LoadAndSaveDeviceCfg**.

3.2.3 Camera Configuration Settings (Only for GigE Camera)

- Interface: public int IMV_GIGE_ForceIpAddress(string pIpAddress, string pSubnetMask, string pGateway);
  - ◇ Description: Modify the IP address of device (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-9 Parameter description

Parameter	Description
pIpAddress [IN]	IP address.
pSubnetMask [IN]	Subnet Mask.
pGateway [IN]	Default Gateway.

    - □ If both **pSubnetMask** and **pGateway** are configured with valid value when calling this function, the valid values shall prevail.
    - □ If both **pSubnetMask** and **pGateway** are configured with **NULL** when calling this function, replace it with subnet mask and gateway of the connected network card during the internal implementation.
    - □ If one of the **pSubnetMask** and **pGateway** is configured with **NULL** and the other is **not NULL**, an error will be returned.
  - ◇ Reference Routine: **ForceIp**.
- Interface: public in IMV_GIGE_GetAccessPermission(ref IMVDefine.IMV_ECameraAccessPermission pAccessPermission);
  - ◇ Description: Obtain the current access rights of the device (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-10 Parameter description

Parameter	Description
pAccessPermission [OUT]	Current access rights of the device.

  - ◇ Reference Routine: None.
- Interface: public int IMV_GIGE_SetAnswerTimeout(UInt32 timeout);
  - ◇ Description: Configure the timeout period for device's response to SDK command (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-11 Parameter description

Parameter	Description
timeout[IN]	Timeout period (unit: s).

◇ Reference Routine: **GigECommunicationControl**.

3.2.4 Unique Features

- Interface: public int IMV_WriteUserPrivateData(IntPtr pBuffer, ref UInt32 pLength);
  - ◇ Description: Write the user-defined data. User can store the 32,768 bytes of user-defined data inside the camera (only for iRAYPLE device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-12 Parameter description

Parameter	Description
pBuffer [IN]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes which has been written.

  - ◇ Reference Routine: None.
- Interface: public int IMV_ReadUserPrivateData(IntPtr pBuffer, ref UInt32 pLength);
  - ◇ Description: Read the user-defined data. User can store the 32,768 bytes of user-defined data inside the camera (only for iRAYPLE device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-13 Parameter description

Parameter	Description
pBuffer [OUT]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes which has been written.

  - ◇ Reference Routine: None.
- Interface: public int IMV_WriteUARTData(IntPtr pBuffer, ref UInt32 pLength);
  - ◇ Description: Write data to the camera serial port register, and history data will be cleared every time user writes (Only for iRAYPLE serialport model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-14 Parameter description

Parameter	Description
pBuffer [IN]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes which has been written.

◇ Reference Routine: None.

- Interface: public int IMV_ReadUARTData(IntPtr pBuffer, ref UInt32 pLength);
  - ◇ Description: Read serial port data from the serial register of camera (only for iRAYPLE camera). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-15 Parameter description

Parameter	Description
pBuffer [OUT]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes which has been written.

  - ◇ Reference Routine: None.

3.3 Register Event Callback

- Interface: public int IMV_SubscribeConnectArg(IMVDefine.IMV_ConnectCallBack proc, IntPtr pUser);
  - ◇ Description: Device connection status event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-16 Parameter description

Parameter	Description
proc [IN]	Device connection status event callback function.
pUser [IN]	User-defined data, can be set to **NULL**.

Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

  - ◇ Reference Routine: **ResumeConnect.**
- Interface: public int IMV_SubscribeParamUpdateArg (IMVDefine.IMV_ParamUpdateCallBack proc, IntPtr pUser);
  - ◇ Description: Parameter update event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-17 Parameter description

Parameter	Description
proc [IN]	Parameter update registration event callback function.

pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **Events.**

- Interface: public int IMV_SubscribeStreamArg(IMVDefine.IMV_StreamCallBack proc, IntPtr pUser);

◇ Description: Stream channel event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-18 Parameter description

Parameter	Description
proc[IN]	Stream channel event callback registration function.
pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **Events.**

- Interface: public int IMV_SubscribeMsgChannelArg (IMVDefine.IMV_MsgChannelCallBack proc, IntPtr pUser);

◇ Description: Notification channel event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-19 Parameter description

Parameter	Description
proc [IN]	Stream channel event callback registration function.
pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **Events.**

3.4 Camera Stream Resource Operation Instruction

3.4.1 Stream Data Configuration

- Interface: public int IMV_SetBufferCount(UInt32 nSize);
  - ◇ Description: Configure the buffer quantity of frame data. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-20 Parameter description

Parameter	Description
nSize [IN]	Buffer quantity.



The configuration cannot be performed during the streaming.

  - ◇ Reference Routine: None.
- Interface: public int IMV_ClearFrameBuffer();
  - ◇ Descrition: Clear the frame data buffer. If succeeded, return **IMV_OK**. If failed, return error code.
  - ◇ Reference Routine: **ClearFrameBuffer**
- Interface: public int IMV_GIGE_SetInterPacketTimeout(UInt32 nTimeout);
  - ◇ Description: configure the internal time of data packet resending (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code. The packet loss resending mechanism under the trigger mode.

Table 3-21 Parameter description

Parameter	Description
nTimeout [IN]	Interval time of data packet (Unit: ms).

  - ◇ Reference Routine: **GigECommunicationControl**.
- Interface: public int IMV_GIGE_SetSingleResendMaxPacketNum(UInt32 maxPacketNum);
  - ◇ Description: Configure the internal time of data packet resending (only for the GigE model device). If succeeded, return **IMV_OK.** If failed, return error code. This function is invalid when **maxPacketNum** is 0.

Table 3-22 Parameter description

Parameter	Description
maxPacketNum [IN]	The max number of data packet resending at one time.

  - ◇ Reference Routine: **GigECommunicationControl**.

- Interface: public int IMV_GIGE_SetMaxLostPacketNum(UInt32 maxLostPacketNum);
  - ◇ Description: Configure the max number of packet loss of the same frame (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-23 Parameter description

Parameter	Description
maxLostPacketNum[IN]	The max number of packet loss.



- This function is invalid when **maxLostPacketNum** is 0.
- Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

  - ◇ Reference Routine: **GigECommunicationControl**.
- Interface: public int IMV_USB_SetUrbTransfer(UInt32 nNum, UInt32 nSize);
  - ◇ Description: Configure the quantity and volume of data block transmission of the U3V device (only for USB model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-24 Parameter description

Parameter	Description
nNum[IN]	The number of data block transmission (range: 5~256).
nSize [IN]	The volume of data block transmission (range:8 KByte ~512 KByte).

  - □ This interface is valid only when the platform is Linux and no driver is adopted, otherwise, it returns the error code of **IMV_NOT_SUPPORT**.
  - □ Data block transmission quantity:

    Range: 5~256 (default value: 64)

    If this value is too small, it will affect the transmission efficiency; When multiple cameras are connected and used together, this value can be appropriately reduced.
  - □ The volume of every data block (Range: 8 KByte ~512 KByte; Default value: 64 KByte).
  - ◇ Reference Routine: None.

3.4.2 Image Collection

- Interface: public int IMV_StartGrabbing();
  - ◇ Description: Acquiring images. If succeeded, return **IMV_OK**. If failed, return error code.
  - ◇ Reference Routine: **Grab**.
- Interface: public int IMV_StartGrabbingEx(UInt64 maxImagesGrabbed, IMVDefine.IMV_EGrabStrategy strategy);
  - ◇ Description: Acquiring images. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-25 Parameter description

Parameter	Description
maxImagesGrabbed [IN]	The max number of frames of acquired images. After the specified number of images is acquired, the acquisition will stop. If the specified number is 0, it means that the acquisition is continuously performed. The parameter of **IMV_StartGrabbing** is 0 by default.
strategy [IN]	Acquisition Strategy (**IMV_StartGrabbing** wil use the **grabStrartegySequential** to acquire image).

  - ◇ Reference Routine: **GrabStrategy.**
- Interface: public bool IMV_IsGrabbing();
  - ◇ Description: Determine whether the device is streaming. If streaming, return True. If not streaming, return **False**.
  - ◇ Reference Routine: **MultipleCamera**.
- Interface: public int IMV_StopGrabbing();
  - ◇ Description: Stop Streaming. If succeeded, return **IMV_OK**. If failed, return error code.
  - ◇ Reference Routine: **Grab**.

3.4.3 Acquire ImageData

- Interface: public int IMV_AttachGrabbing(IMVDefine.IMV_FrameCallBack proc, IntPtr pUser);
  - ◇ Description: Registered frame data callback function (asynchronous acquisition mechanism for frame data). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-26 Parameter description

Parameter	Description
proc [IN]	Frame data information callback function. Do not handle the time-consuming operations in this function, otherwise, it will affect the real-time performance of subsequent frame data.
pUser [IN]	User-defined data, can be set to **NULL**.



- The asynchronous acquisition mechanism and synchronous acquisition mechanism are mutually exclusive, therefore, only one of them can be selected in the system for the same device.
- Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

  - ◇ Reference Routine: **ChunkData**, **Events**, **FrameClone**, **FrameTriggerCount**, **GrabCallback**.

- Interface: public int IMV_GetFrame(ref IMVDefine.IMV_Frame pFrame, UInt32 timeoutMS);
  - ◇ Description: Receive a frame of image (Synchronous acquisition mechanism for frame data). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-27 Parameter description

Parameter	Description
pFrame [OUT]	the frame data information.
timeoutMS[IN]	The timeout value for obtaining a frame of image. The INFINITE mode means that it will continuously wait until a frame of image is received, otherwise, it will stop acquiring (unit: ms).

- □ This interface does not support the multi-thread callings.
- □ The synchronous and asynchronous acquisition mechanism (**IMV_AttachGrabbing**) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.
- □ Acquiring images through releasing the internal cache. Use **IMV_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: **ClearFrameBuffer, Grab.**

- Interface: public int IMV_ReleaseFrame(ref IMVDefine.IMV_Frame pFrame);
  - ◇ Description: Release the image buffer space. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-28 Parameter description

Parameter	Description
pFrame [IN]	the frame data information.

◇ Reference Routine: **ClearFrameBuffer, Grab.**

3.4.4 Operations on Streaming and Image Data

- Interface: public int IMV_CloneFrame(ref IMVDefine.IMV_Frame pFrame, ref IMVDefine.IMV_Frame pCloneFrame);
  - ◇ Description: Frame data deep copy and clone. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-29 Parameter description

Parameter	Description
pFrame [IN]	Clone the source frame data information.
pCloneFrame [OUT]	New frame data information.



Use **IMV_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: **FrameClone.**

- Interface: public int IMV_GetChunkDataByIndex(ref IMVDefine.IMV_Frame pFrame, UInt32 index, ref IMVDefine.IMV_ChunkDataInfo pChunkDataInfo);
  - ◇ Description: Obtain Chunk Data (only for GigE/USB model cameras). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-30 Parameter description

Parameter	Description
pFrame [IN]	The frame data information.

index [IN]	Index ID.
pChunkDataInfo [OUT]	Chunk data information.

◇ Reference Routine: **ChunkData**.

- Interface: public int IMV_GetStatisticsInfo(ref IMVDefine.IMV_StreamStatisticsInfo pStreamStatsInfo);
  - ◇ Description: Obtain statistics information of streaming (after performing **IMV_StartGrabbing** to call the obtaining command). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-31 Parameter description

Parameter	Description
pStreamStatsInfo [OUT]	Stream statistics information.

  - ◇ Reference Routine: **StreamStatistics**.
- Interface: public int IMV_ResetStatisticsInfo();
  - ◇ Description: Reset statistics information of streaming (after performing **IMV_StartGrabbing** to call the resetting command). If succeeded, return **IMV_OK**. If failed, return error code.
  - ◇ Reference Routine: **StreamStatistics**.

3.5 Attribute Operation Instruction

3.5.1 Determination, Acquisition and Setting of Properties

3.5.1.1 Propriety Determination

- Interface: public bool IMV_FeatureIsAvailable(string pFeatureName);
  - ◇ Description: Determine whether the attribute is available. If available, return **True**. If unavailable, return **False**.

Table 3-32 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: None.
- Interface: public bool IMV_FeatureIsReadable (string pFeatureName);
  - ◇ Description: Determine whether the attribute is readable. If readable, return **True**. If unreadable, return **False**.

Table 3-33 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

◇ Reference Routine: None.

- Interface: public bool IMV_FeatureIsWriteable (string pFeatureName);
  - ◇ Description: Determine whether the attribute is writable. If writable, return **True**. If unwritable, return **False**.

Table 3-34 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

◇ Reference Routine: None.

- Interface: public bool IMV_FeatureIsStreamable (string pFeatureName);
  - ◇ Description: Determine whether the attribute is streamable. If streamable, return **True**. If unstreamable, return **False**.

Table 3-35 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

◇ Reference Routine: None.

- Interface: public bool IMV_FeatureIsValid (string pFeatureName);
  - ◇ Description: Determine whether the attribute is valid. If valid, return **True**. If invalid, return **False**.
  - ◇ Reference Routine: **Record.**

3.5.1.2 Attribute Acquisition

- Interface: public int IMV_GetIntFeatureValue(string pFeatureName, ref Int64 pIntValue);
  - ◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-36 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of Integer.

◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetIntFeatureMin (string pFeatureName, ref Int64 pIntValue);

◇ Description: Obtain the configurable min value of the integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-37 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The min value that the integer can be configured.

◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetIntFeatureMax (string pFeatureName, ref Int64 pIntValue);

◇ Description: Obtain the configurable max value of integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-38 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the integer attribute.

◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetIntFeatureInc (string pFeatureName, ref Int64 pIntValue);

◇ Description: Obtain the step size of the integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-39 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	Integer attribute step size.

◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetDoubleFeatureValue (string pFeatureName, ref double pDoubleValue);

◇ Description: Obtain the floating-point attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-40 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of floating point.

◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetDoubleFeatureMin (string pFeatureName, ref double pDoubleValue);
  - ◇ Description: Obtain the configurable min value of the floating-point attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-41 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable min value of the floating-point attribute.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface: public int IMV_GetDoubleFeatureMax (string pFeatureName, ref double pDoubleValue);
  - ◇ Description: Obtain the configurable max value of floating-point attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-42 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the floating-point attribute.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface: public int IMV_GetBoolFeatureValue (string pFeatureName, ref bool pBoolValue);
  - ◇ Description: Obtain the Boolean attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-43 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of Boolean.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface: public int IMV_GetStringFeatureValue (string pFeatureName, ref IMVDefine.IMV_String pStringValue);
  - ◇ Description: Obtain the string attribute value. If succeeded, return IMV_OK. If failed, return error code.

Table 3-44 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

pIntValue [OUT]	The attribute value of String.

◇ Reference Routine: **CommPropAccess**.

- Interface: public int IMV_GetEnumFeatureValue (string pFeatureName, ref UInt64 pEnumValue);

  ◇ Description: Obtain the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-45 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The attribute value of enumeration.

  ◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetEnumFeatureSymbol (string pFeatureName, ref IMVDefine.IMV_String pEnumValueSymbol);

  ◇ Description: Obtain the enumeration symbol value. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-46 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The symbol value of enumeration.

  ◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetEnumFeatureEntryNum (string pFeatureName, ref UInt32 pEntryNum);

  ◇ Description: Obtain the configurable number of the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-47 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEntryNum [OUT]	The configurable quantity of the enumeration attribute value.

  ◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_GetEnumFeatureEntrys (string pFeatureName, ref IMVDefine.IMV_EnumEntryList pEnumEntryList);

  ◇ Description: Obtain the enumeration value list of the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-48 Parameter description

Parameter	Description

pFeatureName [IN]	Attribute name.
pEnumEntryList [OUT]	The list of configurable enumeration value of enumeration attribute.

◇ Reference Routine: **CommPropAccess.**

3.5.1.3 Attribute Setting

- Interface: public int IMV_SetIntFeatureValue(string pFeatureName, Int64 intValue);
  - ◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-49 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
intValue [IN]	The integer attribute value to be configured.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface: public int IMV_SetDoubleFeatureValue (string pFeatureName, double doubleValue);
  - ◇ Description: Configure the floating-point attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-50 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
doubleValue [IN]	Floating point attribute value which is to be configured.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface: public int IMV_SetBoolFeatureValue (string pFeatureName, bool boolValue);
  - ◇ Description: Configure the Boolean attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-51 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
boolValue [IN]	Boolean attribute value which is to be configured.

  - ◇ Reference Routine: **ChunkData.**
- Interface: public int IMV_SetEnumFeatureValue (string pFeatureName, UInt64 enumValue);

◇ Description: Configure the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-52 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
enumValue [IN]	Enumeration attribute value which is to be configured.

◇ Reference Routine: **CommPropAccess.**

- Interface: public int IMV_SetEnumFeatureSymbol (string pFeatureName, string pEnumSymbol);

◇ Description: Configure the enumeration symbol value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-53 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
boolValue [IN]	The symbol value of enumeration which is to be configured.

◇ Reference Routine: **ChunkData**, **CommPropAccess**, **SoftTrigger**, **UserSetControl.**

- Interface: public int IMV_SetStringFeatureValue (string pFeatureName, string pStringValue);

◇ Description: Configure the string attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-54 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pStringValue [IN]	String attribute value which is to be configured.

◇ Reference Routine: **CommPropAccess.**

3.5.2 Execution of Command Attribute

- Interface: public int IMV_ExecuteCommandFeature(string pFeatureName);

◇ Description: Execute the command attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-55 Parameter description

Parameter	Description

pFeatureName [IN]	Attribute name.

◇ Reference Routine: **ChunkData**, **ClearFrameBuffer**, **SoftTrigger**, **UserSetControl**

3.5.3 Attribute Search and Usage

User can search the attribute on the client, and learn the detailed information of the attribute, such as data type, value range, etc.

3.5.3.1 Integer Attribute

If user needs to obtain the pixel height information of the camera, find and click the height attribute. The attribute information of it will be displayed in the below. The data type of the height attribute is int integer as shown in the figure.

Figure 3-1 Height attribute information



User can obtain and modify the found attributes.

```
int ret = IMVDefine.IMV_OK;
Int64 heightValue = 0;
Int64 heightMinValue = 0;
Int64 heightMaxValue = 0;
Int64 heightMaxValue = 0;
Mycamera cam=new Mycamera();

// Obtain attribute value.
```

```
    ret = cam.IMV_GetIntFeatureValue("Height", ref heightValue);
    if (IMVDefine.IMV_OK != ret)
    {
            Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
    }

    // Obtain the configurable min value of attribute. If there is no min value, return the
error code.
    ret = cam.IMV_GetIntFeatureMin("Height", ref heightMinValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
        return ret;
    }

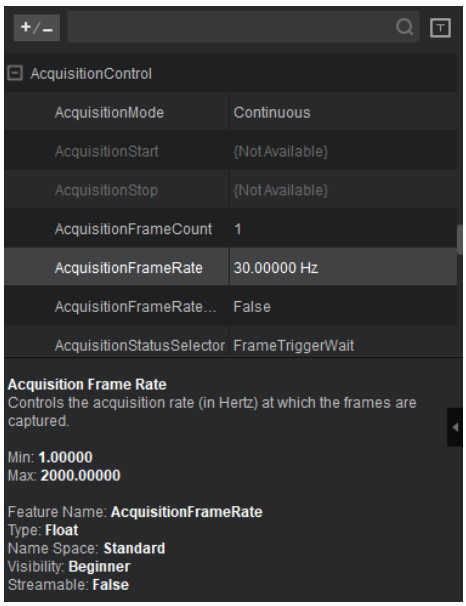
    // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = cam.IMV_GetIntFeatureMax("Height", ref heightMaxValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
        return ret;
    }

    // Obtain the attribute step size. If there is no step-size value, return the error code.
    ret = cam.IMV_GetIntFeatureInc("Height", ref incrementValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature increment value failed! ErrorCode[{ret}]");
        return ret;
    }

    // Configure attribute value.
    ret = cam.IMV_SetIntFeatureValue("Height", heightValue);
```

```
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

3.5.3.2 Floating Point Attribute




```
int ret = IMVDefine.IMV_OK;
double framerate = 0;
double frameratemin = 0;
double frameratemax = 0;
Mycamera cam=new Mycamera();

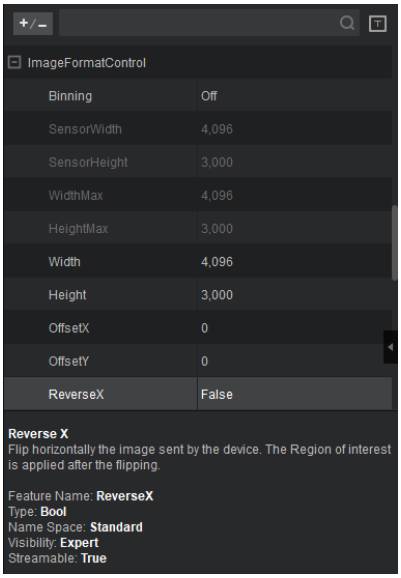
// Obtain attribute value.
ret = cam.IMV_GetDoubleFeatureValue ("AcquisitionFrameRate", ref framerate);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// Obtain the configurable min value of attribute. If there is no min value, return the error code.
```

```
    ret = cam.IMV_GetDoubleFeatureMin("AcquisitionFrameRate", ref frameratemin);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
        return ret;
    }

    // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = cam.IMV_GetDoubleFeatureMax("AcquisitionFrameRate", ref frameratemax);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
        return ret;
    }

    // Configure attribute value.
    ret = cam.IMV_SetDoubleFeatureValue("AcquisitionFrameRate", framerate);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
        return ret;
    }
```

3.5.3.3 Boolean Attribute



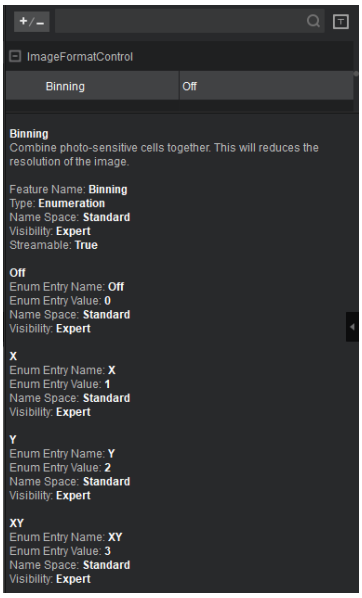

```
  int ret = IMVDefine.IMV_OK;
  bool reversevalue = 0;
Mycamera cam=new Mycamera();

// Obtain attribute value.
ret = cam.IMV_GetBoolFeatureValue ("ReverseX", ref reversevalue);
if (IMVDefine.IMV_OK != ret)
{
      Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
      return ret;
}

// Configure attribute value.
ret = cam.IMV_SetBoolFeatureValue("ReverseX", reversevalue);
if (IMVDefine.IMV_OK != ret)
{
      Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
      return ret;
}
```

3.5.3.4 Enumeration Attribute

Figure 3-2 Enumeration attribute information




```
    int ret = IMVDefine.IMV_OK;
   UInt64 enumValue = 0;
    UInt32 pEntryNum = 0;
    IMVDefine.IMV_String pEnumSymbol=new IMVDefine.IMV_String();
    IMVDefine.IMV_EnumEntryList pEnumEntryList=new
IMVDefine.IMV_EnumEntryList();

   // Obtain the configurable enumeration value list of attribute.
   ret = cam.IMV_GetEnumFeatureValue("Binning", ref pEnumEntryList);
   if (IMVDefine.IMV_OK != ret)
   {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
   }

   //Obtain the number of configurable enumeration values of attribute
   ret = cam.IMV_GetEnumFeatureEntryNum("Binning", ref pEntryNum);
   if (IMVDefine.IMV_OK != ret)
   {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
```

```
        return ret;
    }

    // Obtain the attribute symbol value.
    ret = cam.IMV_GetEnumFeatureSymbol("Binning", ref pEnumSymbol);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
    }

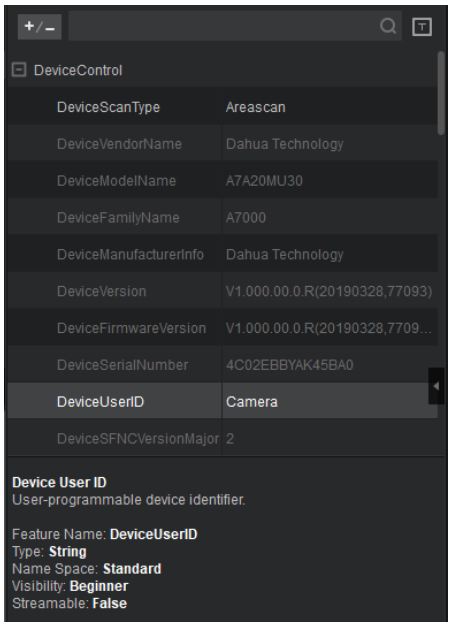
    // Obtain attribute value.
    ret = cam.IMV_GetIntFeatureInc("Binning", ref enumValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
    }

    // Configure attribute symbol value.
    ret = cam.IMV_SetEnumFeatureValue("Binning", pEnumSymbol);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
        return ret;
    }

      // Configure attribute value.
    ret = cam.IMV_SetEnumFeatureValue("Binning", enumValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
        return ret;
    }
```

3.5.3.5 String Attribute

Figure 3-3 String attribute information



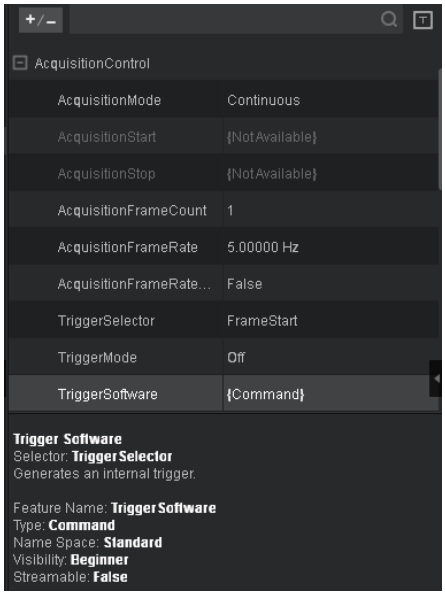

```
  int ret = IMVDefine.IMV_OK;
string id = "";

// Obtain attribute value.
ret = cam.IMV_GetStringFeatureValue ("DeviceUserID", ref id);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// Configure attribute value.
ret = cam.IMV_SetStringFeatureValue("DeviceUserID", id);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

3.5.3.6 Command Attribute

Figure 3-4 Command attribute information




```
int ret = IMVDefine.IMV_OK;
// Execute the attribute value of command.
ret = cam.IMV_ExecuteCommandFeature ("TriggerSoftware");
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Execute Command feature failed! ErrorCode[{ret}]");
    return ret;
}
```

3.6 Image Operation

3.6.1 Video Recording

- Interface: public int IMV_OpenRecord(ref IMVDefine.IMV_RecordParam pstRecordParam);
  - ◇ Description: Start recording. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-56 Parameter description

Parameter	Description
pstRecordParam[IN]	Recording parameter struct.

◇ Reference Routine: **Record.**

- Interface: public int IMV_InputOneFrame(ref IMVDefine.IMV_RecordFrameInfoParam pstRecordFrameInfoParam);
  - ◇ Description: Record a frame of image. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-57 Parameter description

Parameter	Description
pstRecordFrameInfoParam [IN]	Frame information structure for recording.

  - ◇ Reference Routine: None.
- Interface: public int IMV_CloseRecord();
  - ◇ Description: Stop recording. If succeeded, return **IMV_OK**. If failed, return error code.
  - ◇ Reference Routine: **Record.**

3.6.2 Image Operation

- Interface: public int IMV_PixelConvert(ref IMVDefine.IMV_PixelConvertParam pstPixelConvertParam);
  - ◇ Description: Pixel format conversion. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-58 Parameter description

Parameter	Description
pstPixelConvertParam [IN][OUT]	Pixel format conversion parameter structure.

  - □ The supported target pixel format: **gvspPixelRGB8**/ **gvspPixelBGR8**/ **gvspPixelMono8**/ **gvspPixelBGRA8**.
  - □ Convert the raw image data into the pixel format required by the user and stored in the specified memory by the caller.
  - □ When pixel format is **YUV411Packed**, the image width must be divisible by 4.
  - □ When pixel format is **YUV422Packed**, the image width must be divisible by 2.
  - □ When pixel format is **YUYVPacked**, the image width must be divisible by 2.
  - □ Converted image: Data storage starts from the first row at the top, which is the default storage direction of camera data.

◇ Reference Routine: **ImageConvert**, **SaveImageToBmp**

- Interface: public int IMV_FlipImage(ref IMVDefine.IMV_FlipImageParam pstFlipImageParam);

◇ Description: Image flip. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-59 Parameter description

Parameter	Description
pstFlipImageParam [IN][OUT]	Image flip parameter structure.

□ The supported pixel format of the vertical and horizontal image flip: **gvspPixelRGB8**/ **gvspPixelBGR8**/ **gvspPixelMono8**.

□ Convert the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageFlip**

- Interface: public int IMV_RotateImage(ref IMVDefine.IMV_RotateImageParam pstRotateImageParam);

◇ Description: clockwise rotation of the image. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-60 Parameter description

Parameter	Description
pstRotateImageParam[IN][OUT]	Image rotation parameter structure.

□ The supported image format of the 90/180/270 degrees clockwise rotation: **gvspPixelRGB8** / **gvspPixelBGR8** / **gvspPixelMono8**.

□ Rotate the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageRotate**

- Interface: public int IMV_SaveImageToFile(ref IMVDefine.IMV_SaveImageToFileParam pstSaveImageToFileParam);

◇ Description: Save the image. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-61 Parameter description

Parameter	Description
pstSaveImageToFileParam [IN]	Image file saving parameter structure.

□ The supported image format: BMP/JPEG/PNG/TIFF.

□ The supported max height-width size of JPEG format is 65,500.

◇ Reference Routine: **SaveImageToFile**.

4 Conversion method for Image Format of 3rd-party Platform

4.1 Halcon

4.1.1 Mono8 (Monochrome Image)

When image format is Mono8, it can be directly converted to Halcon image object.

It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK.

```
HObject Hobj; //Halcon image object
HOperatorSet.GenImage1Extern(out Hobj,
                             "byte",
                             frame.frameInfo.width,
                             frame.frameInfo.height,
                             frame.pData,
                             IntPtr.Zero);
```

4.1.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into BGR.

Please refer to the **Chapter 3.6.2 Image Operation**.

It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.

```
HObject Hobj; //Halcon image object
HOperatorSet.GenImageInterleaved (out Hobj,
                                  (HTuple) pBGRbuffer,
                                  "bgr", frame.frameInfo.width,
                                  frame.frameInfo.height, 0,
                                  "byte", frame.frameInfo.width,
                                  frame.frameInfo.height,0,0,8,0);
```

4.2 MvpImage Object

4.2.1 Mono8 (Monochrome Image)

For formats such as Mono10 and Mono12, refer to **section 3.6.2** for converting the image format to Mono8. When the camera's image format is Mono8, the captured image data can be directly used to construct the object.

It is assumed here that the **pFrame** is the frame struct of **IMV_Frame** acquired by SDK.

```
MvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImage1(MvpImgDataType.MVP_IMGDTP_U8,
                      (int) pFrame.frameInfo.width,
                      (int)pFrame.frameInfo.height,
                      pData);
```

4.2.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into BGR24.

Please refer to the **Chapter 3.6.2 Image Operation**.

It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK and the **pData** is the BGR data after conversion.

```
HMvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImageInterleaved(pData,
                                EColorSpace.MVP_COLOR_SPACE_BGR,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                MvpImgDataType.MVP_IMGDTP_U8,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                0,
                                -1,
                                0);
```

5 Matched Routine Description

C Routine Path: 【SDK Installation Directory】\Development\Samples\C#\IMV

Table 5-1 Routine description

Routine Name	Routine Description
ChunkData	Obtain block data.
ClearFrameBuffer	Clear image buffer.
CommPropAccess	Common attribute routine.
ConnectSpecCamera	Connect the specified device.
Events	Event processing mechanism.
ForceIp	Modify the IP address of device (only for the GigE model device).
FrameClone	Clone frame.
FrameTriggerCount	Frame trigger statistic.
GigECommunicationControl	GigE Communication Control (only for the GigE model device).
Grab	Image Capturing in free-run mode (active streaming).
GrabCallback	Streaming callback.
GrabStrategy	Image acquisition strategy.
ImageConvert	Image format conversion.
ImageFlip	Image flipping.
ImageRotate	Clockwise rotation of the image.
LineTrigger	Image acquisition in external trigger mode (callback to acquire).
LoadAndSaveDeviceCfg	Save and load the attribute configuration file of device.
MultipleCamera	Multi-camera acquiring.
ResumeConnect	Reconnect device after disconnection.
SaveImageToBmp	Save the BMP image.
SaveImageToFile	Save image to the PC.
SoftTrigger	Acquisition in software trigger mode (callback to acquire).
StreamStatistics	Image acquisition statistics.
UnicastMode	Single-cast function (only for GigE model device).
UserSetControl	Save and load the configuration of device.

6 FAQ (Frequently Asked Question)

6.1 Only 8 Images are Obtained by Using the C# Interface.

Reason

After acquiring the images using C# interface, the images are not released resulting in the buffer memory being filled and preventing the acquisition of new images.

Solution

Every time the frame is acquired, the buffer memory shall be release by calling the interface of **IMV_ReleaseFrame** as shown in the figure below.

Figure 6-1 IMV_ReleaseFrame

```
//get frame
IMVDefine.IMV_Frame frame = new IMVDefine.IMV_Frame();
res = cam.IMV_GetFrame(ref frame, 1000);
if (res != IMVDefine.IMV_OK)
{
    Console.WriteLine("Get frame failed! ErrorCode:[{0}]", res);
    break;
}
```

```
// Free image buffer
res = cam.IMV_ReleaseFrame(ref frame);
if (res != IMVDefine.IMV_OK)
{
    Console.WriteLine("Release frame failed! ErrorCode:[{0}]", res);
    break;
}
```